E0 298: Linear Algebra and Applications

Assignment 6 Solutions

Trishit Mukherjee

M.Tech AI 2023-25

November 17, 2024

1 Regression

Given a data matrix $\mathbf{X} \in \mathbb{R}^{n \times m}$ containing n samples with m features:

1.1 Unconstrained

1. **Multilinear Regression -**

$$\ell_1(\beta) = \|y - X\beta\|_2^2$$
$$\underset{\beta \in \mathbb{R}^{m+1}}{\arg\min}\ \ell_1(\beta) \tag{P}$$
$$\text{Solve for } \beta^* : (X^\top X)\beta^* = X^\top y \Rightarrow \beta^* = (X^\top X)^{-1} X^\top y = X^\dagger y$$

2. **Generalised Regression with polynomial kernel -**

$$\phi_2(X) = Z_2;\ \phi_2 : \mathbb{R}^{n \times 2} \to \mathbb{R}^{n \times 5}$$
$$\phi_3(X) = Z_3;\ \phi_3 : \mathbb{R}^{n \times 2} \to \mathbb{R}^{n \times 9}$$
$$\phi_4(X) = Z_4;\ \phi_4 : \mathbb{R}^{n \times 2} \to \mathbb{R}^{n \times 14}$$

$$\ell_2(\beta) = \|y - Z\beta\|_2^2;$$
$$\underset{\beta \in \mathbb{R}^{k+1}}{\arg\min}\ \ell_2(\beta) \tag{Q}$$
$$\text{Solve for } \beta^* : (Z^\top Z)\beta^* = Z^\top y \Rightarrow \beta^* = (Z^\top Z)^{-1} Z^\top y = Z^\dagger y$$

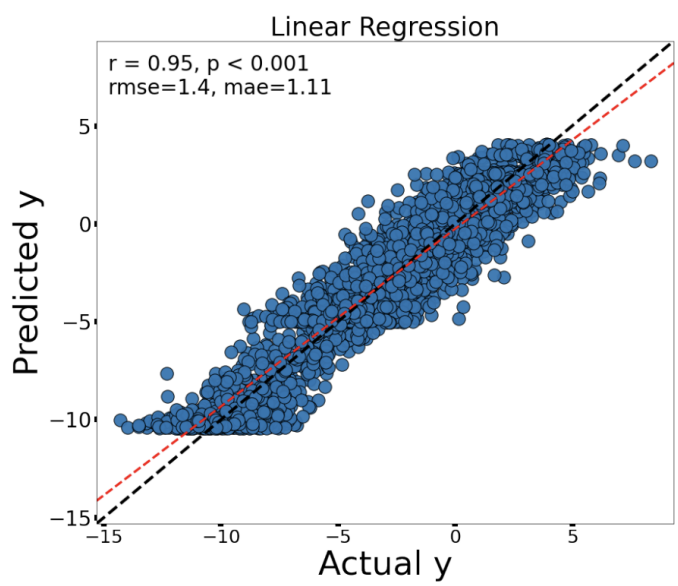

Figure 1: Correlation plot y_{pred} vs y_{act}. The x-axis is the actual target, and the y-axis is the predicted target. The red line shows the line of best fit. The black line is the $y_{pred} = y_{act}$ line (prediction by oracle)

1.2 (Linear) Constrained

1.

$$\ell_3(\beta) = \frac{1}{2}\|y - X\beta\|_2^2$$

$$\arg\min_{\beta \in \mathbb{R}^{m+1}} \ell_3(\beta) \tag{R}$$

$$\text{subject to } A\beta = b$$

$$\begin{aligned}
\ell_3(\beta) &= \frac{1}{2}\beta^\top X^\top X\beta - \langle X\beta, y\rangle + \frac{1}{2}\|y\|_2^2 \\
&= \frac{1}{2}\beta^\top Q\beta - \langle \beta, X^\top y\rangle + c; \quad Q = X^\top X \in \mathcal{S}_+^n,\ c = \frac{1}{2}\|y\|_2^2 \\
\nabla\ell_3(\beta) &= Q\beta - X^\top y
\end{aligned}$$

$$\text{Solve for } \beta^* : \nabla\ell_3(\beta^*) = A^\top\lambda \Rightarrow Q\beta^* - A^\top\lambda = X^\top y \quad \& \quad A\beta^* = b$$

$$\text{Solve for the following system of linear equations : } \begin{pmatrix} A & 0 \\ X^\top X & -A^\top \end{pmatrix} \begin{pmatrix} \beta^* \\ \lambda \end{pmatrix} = \begin{pmatrix} b \\ X^\top y \end{pmatrix}$$

2 Mathematical Formulation of PCA

Given a data matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$ containing n samples with d features:

2.1 Data Centering

Given the data matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$ where n is the number of samples and d is the number of features:

The centering matrix $\mathbf{H} \in \mathbb{R}^{n \times n}$ is defined as:

$$\mathbf{H} = \mathbf{I}_n - \frac{1}{n}\mathbf{1}_n\mathbf{1}_n^\top$$

where:

- $\mathbf{I}_n$ is the $n \times n$ identity matrix
- $\mathbf{1}_n$ is an n-dimensional vector of ones

The centered data matrix $\mathbf{X}_c$ is obtained by:

$$\mathbf{X}_c = \mathbf{H}\mathbf{X}$$

The mean vector $\boldsymbol{\mu} \in \mathbb{R}^d$ can be expressed as:

$$\boldsymbol{\mu} = \frac{1}{n}\mathbf{X}^\top\mathbf{1}_n$$

Properties:

- $\mathbf{H}$ is symmetric: $\mathbf{H} = \mathbf{H}^\top$
- $\mathbf{H}$ is idempotent: $\mathbf{H}^2 = \mathbf{H}$
- $\mathbf{H}\mathbf{1}_n = \mathbf{0}_n$
- $\text{rank}(\mathbf{H}) = n - 1$

2.2 Covariance Matrix

Compute the sample covariance matrix:

$$\mathbf{C} = \frac{1}{n-1}\mathbf{X}_c^\top\mathbf{X}_c \in \mathbb{R}^{d \times d}$$

2.3 Eigendecomposition

The covariance matrix $\mathbf{C}$ is symmetric positive semidefinite, allowing eigendecomposition:

$$\mathbf{C} = \mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^\top$$

where:

- $\mathbf{V} = [\mathbf{v}_1, \ldots, \mathbf{v}_d] \in \mathbb{R}^{d \times d}$ is the matrix of eigenvectors
- $\boldsymbol{\Lambda} = \text{diag}(\lambda_1, \ldots, \lambda_d)$ contains eigenvalues in descending order
- $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_d \geq 0$

2.4 Dimensionality Reduction

Select the top k eigenvectors to form the projection matrix:

$$\mathbf{W}_k = [\mathbf{v}_1, \ldots, \mathbf{v}_k] \in \mathbb{R}^{d \times k}$$

2.5 Projection and Reconstruction

The projection of centered data onto the principal components:

$$\mathbf{Z} = \mathbf{X}_c \mathbf{W}_k \in \mathbb{R}^{n \times k}$$

The reconstruction of the original data:

$$\hat{\mathbf{X}} = \mathbf{Z}\mathbf{W}_k^\top + \mathbf{1}\boldsymbol{\mu}^\top$$

2.6 Reconstruction Error

The mean squared reconstruction error:

$$\text{MSE} = \frac{1}{nd}\|\mathbf{X} - \hat{\mathbf{X}}\|_F^2$$

The Peak Signal-to-Noise Ratio (PSNR):

$$\text{PSNR} = 20\log_{10}(\text{MAX}_{\text{pixel}}) - 10\log_{10}(\text{MSE})$$

where $\text{MAX}_{\text{pixel}}$ is the maximum possible pixel value (1.0 for normalized images; 255 otherwise).

3 Stationary dist. of Markov Chains

3.1 Direct Method (Linear System Solution)

Let $P \in \mathbb{R}^{n \times n}$ be the transition matrix. The direct method solves: $\begin{cases} \pi^\top P = \pi^\top \\ \pi^\top \mathbf{1}_n = 1 \end{cases}$

Note: -

$$\pi^\top P = \pi^\top \Leftrightarrow P^\top \pi = \pi \Leftrightarrow (P^\top - \mathbf{I}_n)\pi = \mathbf{0}_n$$

This is same as solving a system of homogeneous linear equations

$$Ax = b;\ A = P^\top - \mathbf{I}_n,\ x = \pi,\ b = \mathbf{0}_n$$

But, this system has infinitely many solutions as $(P^\top - \mathbf{I}_n)$ is not invertible

$$1 \in \rho(P^\top) \Leftrightarrow 0 \in \rho(P^\top - \mathbf{I}_n) \Leftrightarrow det(P^\top - \mathbf{I}_n) = 0$$

To get around this problem use the constraint $\pi^\top \mathbf{1}_n = 1$.

- Replace any one of the rows (say the i-th) of $P^\top - \mathbf{I}_n$ by $\mathbf{1}_n$.
- Replace the corresponding 0 (i-th) in $\mathbf{0}_n$ by 1.
- This new matrix say P', is invertible and thus a unique solution exists. Solve using `numpy.linalg`

Complexity Analysis

- **Time Complexity**: $\mathcal{O}(n^3)$ using Gaussian elimination
- **Space Complexity**: $\mathcal{O}(n^2)$
- Single computation, non-iterative

3.2 Power Method (Iterative Approach)

The power method iteratively computes : ${\pi^{(k+1)}}^\top = {\pi^{(k)}}^\top P$ until convergence, i.e., $\left\|\pi^{(k+1)} - \pi^{(k)}\right\|_1 < \epsilon;\ \epsilon \to 0$, where k is the iteration number.

Complexity Analysis

- **Per Iteration** – Matrix-vector multiplication: $\mathcal{O}(n^2)$
- **Total Time Complexity**: $\mathcal{O}(i \cdot n^2)$, where i is the number of iterations.
- **Space Complexity**: $\mathcal{O}(n^2)$ for P and $\mathcal{O}(n)$ for $\pi^{(k)}$

3.3 Comparative Analysis

1. **Efficiency Conditions**

   - Power method is more efficient when $i < n$
   - Direct method is more efficient when $i > n$

   Mathematically :
   Power Method Cost $<$ Direct Method Cost $\iff i \cdot n^2 < n^3 \iff i < n$

2. **Space Complexity Trade-off**

$$\begin{aligned} &\text{Direct Method:} \quad \mathcal{O}(n^2) \\ &\text{Power Method:} \quad \mathcal{O}(n^2) \text{ for } P + \mathcal{O}(n) \text{ for vectors} \end{aligned}$$

4 Spectral Norm of a Matrix

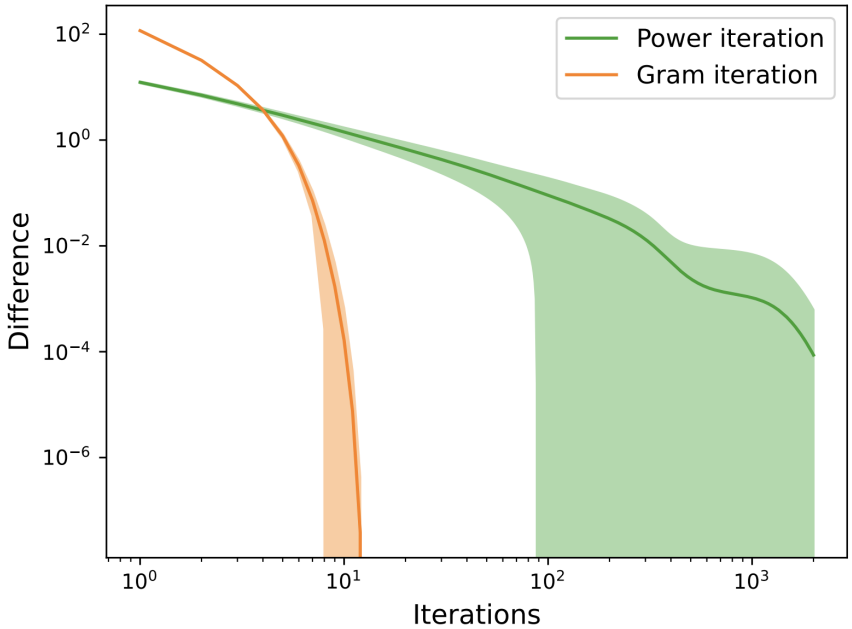

Figure 2: Variance in convergence

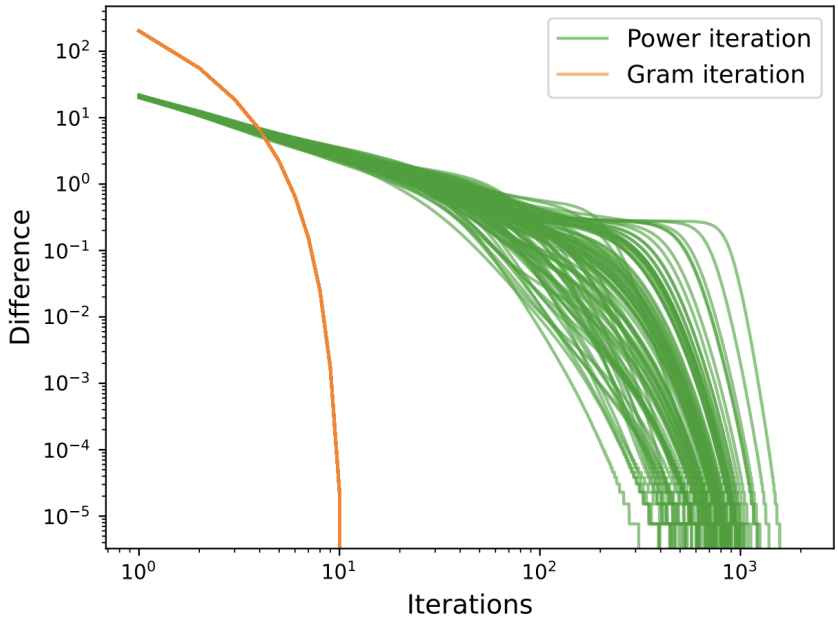

Figure 3: Convergence paths for different runs

1. Gram method is clearly deterministic whereas Power method is stochastic.
2. Gram iteration converges much faster.
3. Power iteration also gives the left and right singular vectors with spectral norm and this can be used to perform SVD (or eigenvectors and eventually EVD if exists).

Algorithm 1 Power_iteration_sym(G, N_{iter})

Inputs matrix: G, number of iterations: N_{iter}
Initialization: draw a random vector u
for $1 \dots N_{\text{iter}}$ **do**
    $v \leftarrow Gu/\|Gu\|_2$
    $u \leftarrow G^* v/\|G^* v\|_2$
end for
$\rho \leftarrow (Gu)^* v$
return ρ, u

Algorithm 2 Min_Max_EigenValue(G, N_{iter})

Inputs matrix: G, number of iterations: N_{iter}
Initialization: $\lambda_{max}, \lambda_{min} \leftarrow$ None, None
$\rho_1, u \leftarrow$ Power_iteration_sym(G, N_{iter})
if $u^\top A u < 0$ **then**
    $\lambda_{min} = -\rho_1$
    $\rho_2, _ \leftarrow$ Power_iteration_sym($G + \rho_1 I, N_{\text{iter}}$)
    $\lambda_{max} = \rho_1 - \rho_2$
else
    $\lambda_{max} = \rho_1$
    $\rho_2, _ \leftarrow$ Power_iteration_sym($G - \rho_1 I, N_{\text{iter}}$)
    $\lambda_{min} = \rho_2 - \rho_1$
end if
return $\lambda_{min}, \lambda_{max}$